

Sztuczna Inteligencja – laboratorium nr 7 - RAG

1. LLM vs RAG vs Agent AI vs MCP

LLM – Large Language Model (Duży Model Językowy)

LLM to fundament całej reszty. To model wytrenowany na ogromnych zbiorach tekstu, który „uczy się” przewidywać, jakie słowo powinno pojawić się następane. Dzięki temu potrafi pisać, tłumaczyć, streszczać i rozumować w języku naturalnym.

Ograniczenie: LLM wie tylko to, co zobaczył podczas treningu. Nie potrafi sprawdzić aktualnych informacji ani wykonać żadnych działań w świecie zewnętrznym.

RAG – Retrieval-Augmented Generation (Generowanie wspomagane wyszukiwaniem)

RAG rozwiązuje problem „nieaktualnej wiedzy”. Zanim LLM odpowie na pytanie, system najpierw przeszukuje wskazaną bazę wiedzy (dokumenty firmowe, stronę internetową, bazę danych) i wstrzykuje znalezione fragmenty do kontekstu modelu. LLM odpowiada więc na podstawie świeżych, konkretnych danych – nie tylko tego, czego uczył się miesiące temu.

Uproszczony przepływ: pytanie użytkownika → wyszukanie w bazie wiedzy → przekazanie wyników do LLM → odpowiedź.

Agent AI

Agent to LLM wyposażony w zdolność do planowania i działania w pętli. Zamiast odpowiadać jednorazowo, agent:

1. Analizuje cel
2. Układa plan kroków
3. Wykonuje działania (np. wyszukuje w internecie, uruchamia kod, wysyła e-mail)
4. Obserwuje wynik
5. Powtarza, aż zadanie zostanie ukończone

Agent może np. samodzielnie zarezerwować lot, sprawdzić kalendarz i wysłać potwierdzenie – wykonując dziesiątki kroków bez udziału człowieka.

MCP – Model Context Protocol

MCP to standardowy protokół komunikacji między modelem AI a zewnętrznymi narzędziami i źródłami danych. Zamiast każdego razu pisać osobną integrację z Gmailem, GitHubem czy bazą danych, MCP definiuje jeden wspólny interfejs. Każde narzędzie udostępniające serwer MCP może być użyte przez dowolny kompatybilny model – bez dodatkowego kodu łączącego.

Podsumowanie

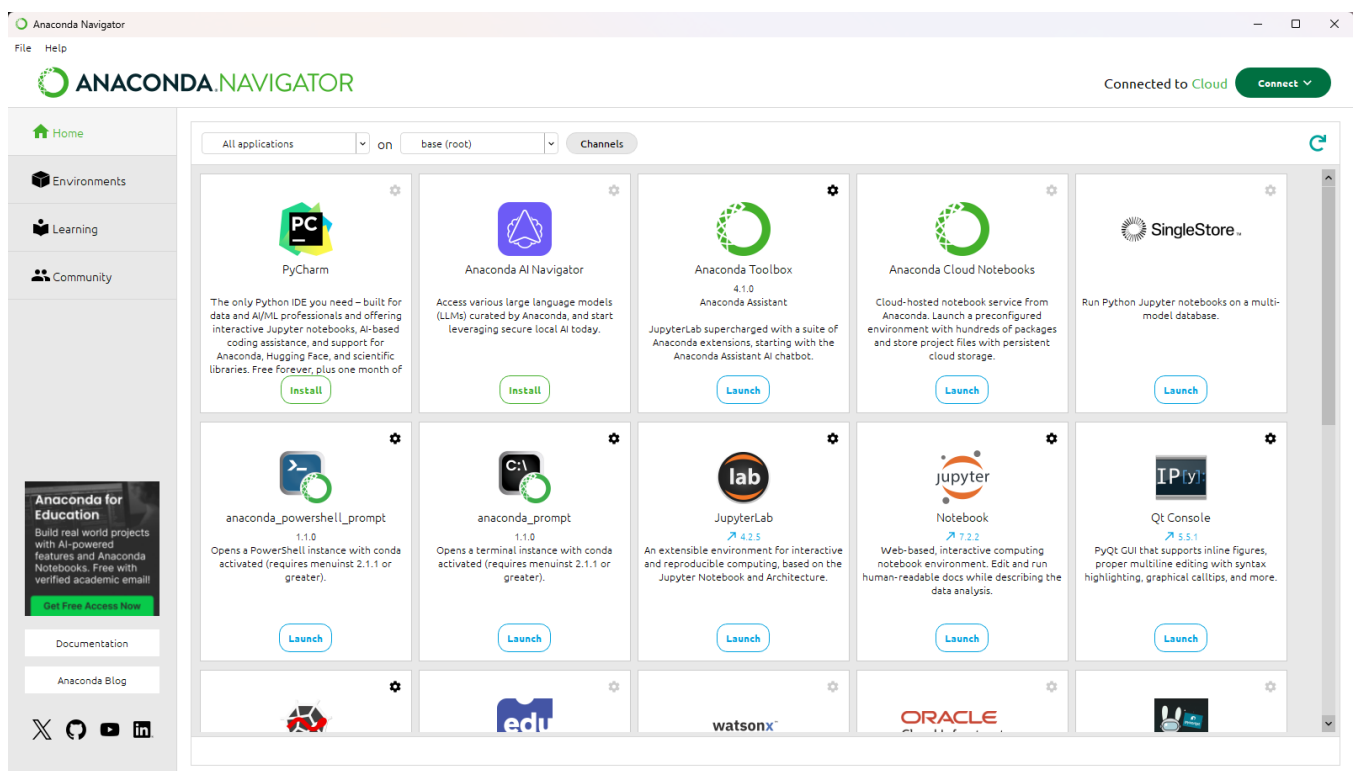
Technologia	Co robi
LLM	Rozumie i generuje tekst na podstawie wiedzy z treningu
RAG	Uzupełnia LLM o aktualne dane z zewnętrznych źródeł
Agent AI	Planuje i wykonuje wieloetapowe zadania w pętli
MCP	Standaryzuje sposób, w jaki modele łączą się z narzędziami i danymi

2. Ćwiczenie 1. Przygotowanie nowego środowiska Anaconda do pracy z prostym RAG.

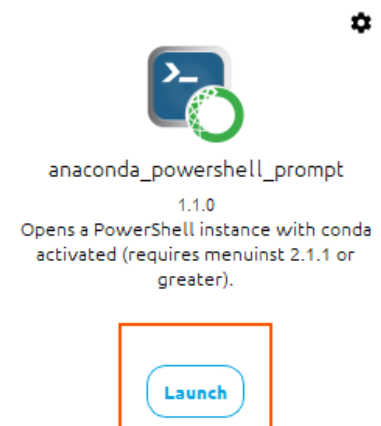
Instalację Anaconda można oczywiście pominąć jeśli jest zainstalowana, ale środowisko wirtualne będzie **NOWE!**

Zainstaluj **Anaconda** ze strony: <https://www.anaconda.com/>.

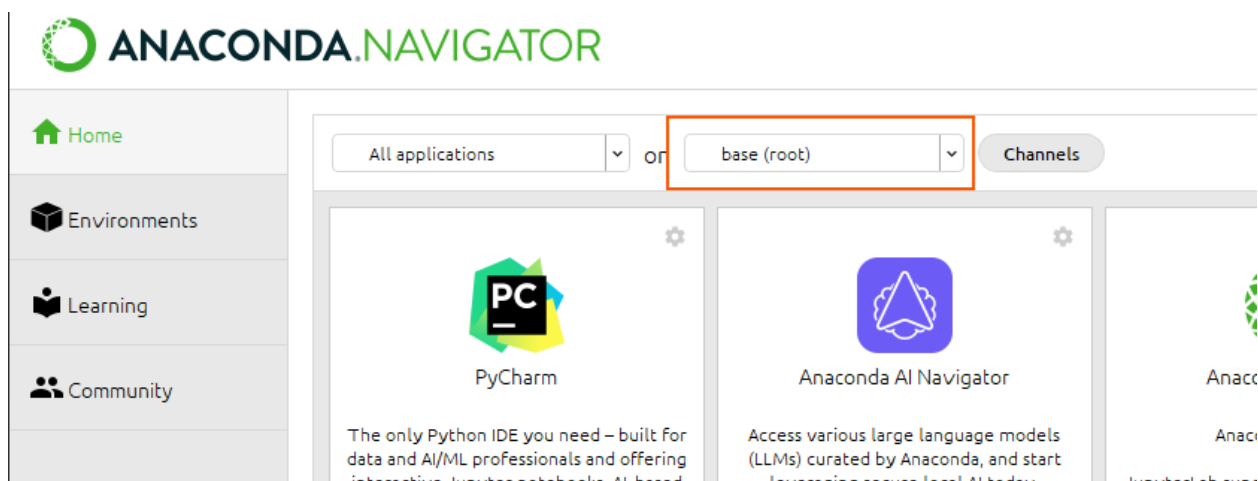
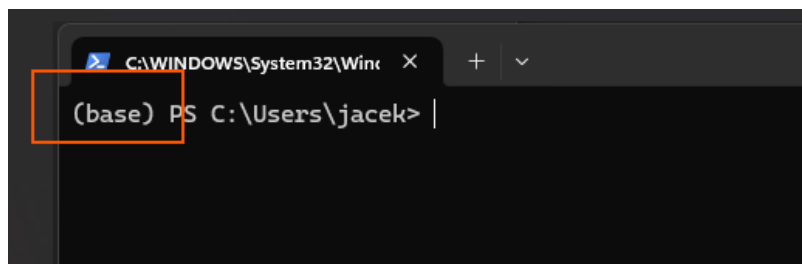
Po zainstalowaniu dostaniemy:



Uruchom następnie narzędzie:



Dostaniemy terminal do pracy z podstawowym wirtualnym środowiskiem Anacondy:



Jednak to środowisko domyślnie nie ma wymaganych bibliotek i pakietów potrzebnych do pracy z modelami LLM i prostym RAG.

Dlatego stworzymy nowe dedykowane.

Praca w wielu wirtualnych środowiskach pozwala nam na pracę przy wielu różnych projektach, które wymagają do pracy innych bibliotek, często w różnych wersjach:

llms2	☒	accelerate	
rag_cpu	☒	asttokens	The asttokens module annotates python abstract syntax trees (asts) with the positions of tokens an
rag_gpu	☒	backports.zstd	Backport of compression.zstd
rag_gpu2	☒	blas	Linear algebra package
	☒	brotli	Brotli compression format
	☒	brotli-python	Brotli compression format
	☒	bzip2	High-quality data compressor
	☒	ca-certificates	Certificates for use with other packages.

Z moodle należy ściągnąć plik o nazwie lub [env_gpu.yml](#) ([env_cpu.yml](#) - jeśli nie mamy GPU, bardzo niezalecane praca kilkanaście razy wolniejsza) i skopiować go do nowo utworzonego katalogu w katalogu użytkownika np.:

```
PS C:\Users\jacek\__LLM\Labs\Lab7RAG>
```

Zanim go uruchomimy jednak musimy dostosować w nim wersję sterowników CUDA, by były zgodne z tym co mamy w komputerze.

Zawartość pliku [env_gpu.yml](#):

```
channels:
  - pytorch
  - nvidia
  - conda-forge
  - defaults

dependencies:
  - python=3.10
  - pip
  - pytorch=2.3.1
  - torchvision
  - torchaudio
  - pytorch-cuda=12.1 # dopasuj do sterownika NVIDIA
  - nin:
    - transformers==4.43.3
    - accelerate==0.33.0
    - safetensors
    - sentencepiece
    - tokenizers>=0.19.0
    - faiss-cpu
    - pymupdf
    - sentence_transformers
    - iprogress
    - ipywidgets
    - numpy
    - scipy
    - tqdm
    - huggingface-hub
```

Uruchom plik [cuda_check.ipynb](#):

```
[1]: # Sprawdzanie CUDA:

import torch

print(torch.cuda.is_available())
if (torch.cuda.is_available()):
    print(torch.cuda.get_device_name(0)) # nazwa karty
    print(torch.version.cuda)
else:
    print("brak")

True
NVIDIA GeForce RTX 4090
12.1
```

Wersję CUDA należy wpisać do pliku [env_gpu.yml](#).

Plik ten pozwoli utworzyć nowe środowisko. Zawiera informacje o wszystkich potrzebnych zasobach do pracy z modelami LLM i prostym RAG.

```
(base) PS C:\Users\jacek\__LLM\Labs\Lab7RAG> conda env create -f env_gpu.yml
```

Menadżer pakietów conda zainstaluje nam wszystkie zawarte w pliku biblioteki i stworzy nam nowe wirtualne środowisko.

Środowisko następnie należy aktywować:

```
(base) PS C:\Users\jacek\__LLM\Labs\Lab7RAG> conda activate rag_gpu
```

```
(base) PS C:\Users\jacek\__LLM\Labs\Lab7RAG> conda activate rag_gpu  
(rag_gpu) PS C:\Users\jacek\__LLM\Labs\Lab7RAG>
```

Gotowe. Możemy teraz popracować z modelami LLM i prostym RAG.

3. Ćwiczenie 3. Wykorzystanie modeli LLM w RAG.

W katalogu projektu utwórz katalogi:

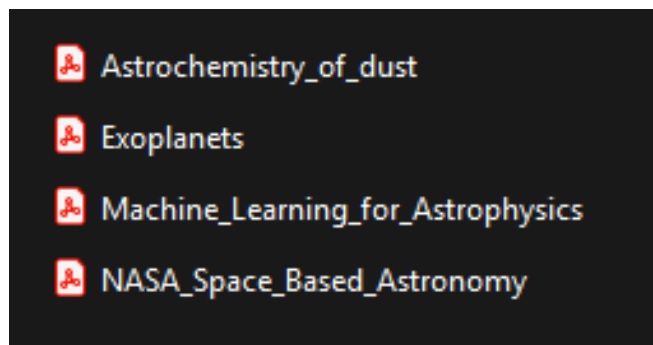
```
mkdir -p vec_db  
mkdir -p knowledge
```

W pierwszym będziemy przechowywać embeddingi naszych tokenów w postaci bazy wektorowej, w drugim będziemy przechowywać wiedzę dla naszego modelu.

Uruchom teraz narzędzie JupyterLab w nowo utworzonym środowisku.

Wykorzystamy model LLM jako mózg w systemie RAG.

Naszym celem będzie zbudowanie narzędzia, które będzie odpowiadać na podstawie załączonych do projektu PDF:



Do zrobienia:

1. Ściągnij pliki PDF z moodle i umieść w katalogu *knowledge*.
2. Z moodle pobierz *plik lab7rag.ipynb* z systemem RAG. W kodzie masz opisane dokładnie co zostało zaimplementowane. Obowiązkowo zapoznaj się ze wszystkimi komentarzami.
3. Uruchom system ☺

ZADANIE:

1. Wykorzystaj ten system by odpowiadał na podstawie innej wiedzy – zamień PDF na inne.
2. Zamiast modelu "microsoft/Phi-3.5-mini-instruct" użyj lokalnego modelu z ollama. Pozwoli to na eksperymentowanie z różnymi modelami w bardzo prosty i darmowy sposób.

WAŻNE!!!!

Dobrze skonfigurowany model powinien opierać się tylko na kontekście i w razie braku odpowiednich informacji powinien zwracać takie odpowiedzi:

Pytanie: Dlaczego wykrywanie małych egzoplanet jest trudniejsze niż dużych?

Odpowiedź:

Kontekst nie zawiera bezpośredniej odpowiedzi na pytanie, dlaczego wykrywanie małych egzoplanet jest trudniejsze niż dużych. Jednakże, kontekst wspomina o trudnościach związanych z obrazowaniem egzoplanet: "Imaging remains highly challenging because of the relative proximity between star and planet, and the enormous contrast ratio (a factor of 10^8 or more) between the visible light from the star and that reflected from, or emitte".

Źródła:

[1] Exoplanets.pdf — strona 7

ch exoplanet masses as small as that of Earth. (I) 46. "Planet–planet eclipse and the Rossiter–McLaughlin effect of a multiple transiting system," T. Hirano, N. Narita, B. Sato et al., ApJ 759, L36–L40...

[2] Exoplanets.pdf — strona 11